

AI BUILD LAB · PRODUCT ENGINEERING

AI Build Lab - Tools I Needed, Built and Shipped

Personally experienced problems become requirements, architecture, acceptance criteria, tested products, releases, and operations; AI amplifies implementation but does not own context or acceptance.

EVIDENCE STATE Ongoing	PERIOD 2024 - present
PORTFOLIO TIER AI Build Lab	PRIMARY CAPABILITY Product Engineering with AI

One combined case connects a local-first knowledge system, a multi-CLI desktop application, and a Daegu bus-information application through the same problem-to-product loop.

Problem and system boundary

01 / PROBLEM TO PRODUCT

Problem to product

Connect friction to requirements, architecture, acceptance, tests, releases, and operations.

PROBLEM

Recurring friction needed to become reusable product requirements rather than one-off scripts.

One combined case connects a local-first knowledge system, a multi-CLI desktop application, and a Daegu bus-information application through the same problem-to-product loop.

PUBLIC BOUNDARY

Never expose private knowledge-system source data or claim unverified user or productivity metrics.

AI amplifies implementation; people own context, architecture, acceptance criteria, review, and release decisions.

EVIDENCE STATE

Ongoing / multi-cli-work v1.24: one project opens CLI and agent sessions alongside git status and worktrees. Local paths are blurred.

Owned decisions and implementation

MY ROLE

Owned problem context, requirements, architecture, acceptance criteria, tests, releases, and operating decisions while using AI to amplify implementation.

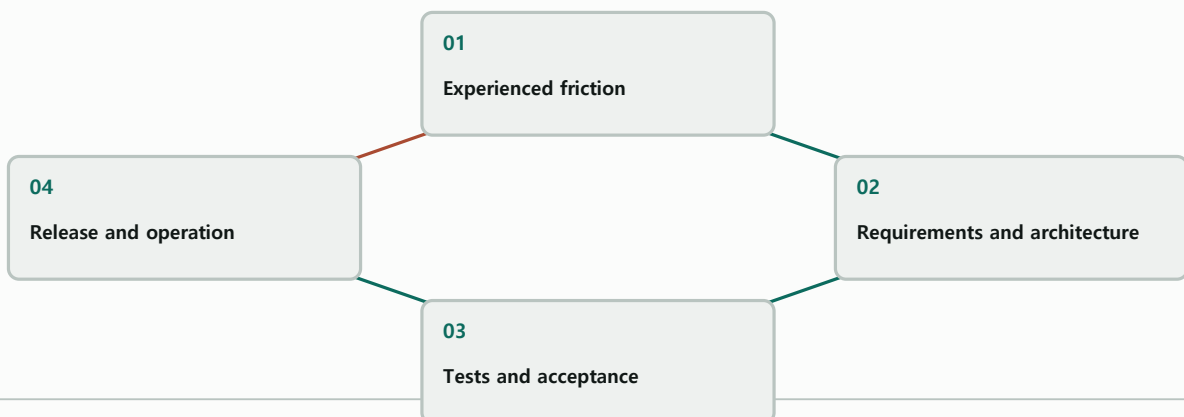
02 / HUMAN-AI BOUNDARY

Human-AI boundary

AI amplifies implementation; people own context, acceptance, and release decisions.

SYSTEM RELATIONSHIP

Product loop from friction to operation



Explanatory system relationship diagram; it is not photographic or experimental evidence.

IMPLEMENTATION TECHNOLOGIES

Electron / TypeScript / Node.js / Cloudflare / GitHub Actions / Agent workflows

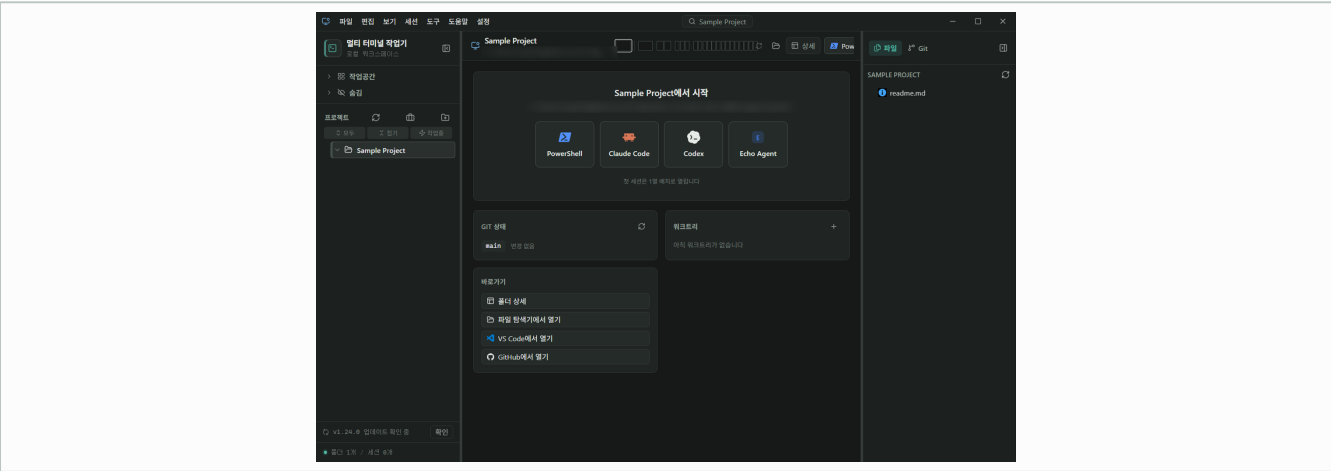
Verification and evidence ledger

03 / PUBLIC PRODUCT PROOF

Public product proof

Use public repositories, automated tests, and release artifacts as observable evidence.

IMAGE / PUBLICLY INSPECTABLE



REGISTERED PUBLIC EVIDENCE

REPOSITORY / PUBLICLY INSPECTABLE multi-cli-work-repository Public source and release history; registered as repository evidence, not visual media. Open public link
REPOSITORY / PUBLICLY INSPECTABLE daegu-bus-repository Public source history; registered as repository evidence, not visual media. Open public link
IMAGE / PUBLICLY INSPECTABLE ai-build-lab-lead-01 Approved derivative; caption in portfolio data.
IMAGE / PUBLICLY INSPECTABLE ai-build-lab-gallery-01 Approved derivative; caption in portfolio data.
IMAGE / PUBLICLY INSPECTABLE ai-build-lab-gallery-02 Approved derivative; caption in portfolio data.

Role, team result, and limits

04 / PRIVACY AND METRIC BOUNDARY

Privacy and metric boundary

Exclude private knowledge data and unverified user or productivity metrics.

MY ROLE

Own context, architecture, acceptance criteria, tests, and releases.

Owned problem context, requirements, architecture, acceptance criteria, tests, releases, and operating decisions while using AI to amplify implementation.

TEAM RESULT

Public repositories and release artifacts are observable product results; no user-count, productivity, or maintainability metrics are claimed.

Screens from a released multi-cli-work build, together with the public repositories, tests, and releases of both applications, provide public-safe evidence; private knowledge data is excluded.

LIMITATIONS

Never expose private knowledge-system source data or claim unverified user or productivity metrics.

multi-cli-work v1.24: one project opens CLI and agent sessions alongside git status and worktrees. Local paths are blurred.

COLLABORATION BOUNDARY

AI amplifies implementation; people own context, architecture, acceptance criteria, review, and release decisions.

One combined case connects a local-first knowledge system, a multi-CLI desktop application, and a Daegu bus-information application through the same problem-to-product loop.

Recap and joint-development invitation

Personally experienced problems become requirements, architecture, acceptance criteria, tested products, releases, and operations; AI amplifies implementation but does not own context or acceptance.

PRIMARY CAPABILITY

Product Engineering with AI

REGISTERED PUBLIC EVIDENCE

[multi-cli-work repository](#)

[Daegu bus app repository](#)

WHAT TO BRING

Share the problem, data or sensors, target validation, and schedule. We can define coordinate and evidence boundaries together.

AI amplifies implementation; people own context, architecture, acceptance criteria, review, and release decisions.

uiop3847@naver.com

rafaam11.github.io